

پروژه پایانی سطح ۰۱ · قسمت صفر

مقدمه و ابزارها

پیش از شروع هر سه پروژه: داده را از کجا بیاورم و چه ابزاری لازم دارم؟

فصل ۱
از داده تا سیگنال؛ نقشه هر سه پروژه
صفحه ۵

فصل ۲
API، JSON و پنل Network
صفحه ۸

فصل ۳
تله‌های داده: متن، واحد، ترتیب و حرف عربی
صفحه ۱۲

فصل ۴
چهار چیز تازه: try، مرتب کردن، جدول و فایل
صفحه ۱۵

فصل ۵
انحراف معیار و امتیاز z؛ بدون numpy
صفحه ۱۹

فصل ۶
سلول تست، چک‌لیست و وقتی گیر می‌کنی
صفحه ۲۲

حسین حاجی آقا بزرگی

پیش از شروع

جعبه‌ابزاری که هر سه پروژه با آن ساخته می‌شوند

سه پروژه پایانی سه سؤال متفاوت دارند، ولی مسیرشان یکی است: داده واقعی را از API عمومی یک سایت می‌گیری، در list و dict مرتبش می‌کنی، رویش حساب می‌کنی، یک سیگنال می‌دهی و گزارش می‌نویسی. این دفترچه همان قسمت مشترک است: API و JSON، دو ابزار آماده، تله‌های داده، چهار چیز تازه و دو مفهوم آماری. وقتی این قسمت را تمام کنی، هر کدام از سه پروژه را که انتخاب کنی، ابزارش را در دست داری.

قانون‌های پروژه

- فقط ابزارهای سطح ۱: `list`، `dict`، `tuple`، `set`، متدهای رشته، `for`، `while`، `if`، `sorted`، `len`، `sum`، `round` و f-string.
- **numpy و pandas ممنوع.** میانگین، انحراف معیار، بازده و سقف و کف را خودت با حلقه حساب می‌کنی.
- همه داده‌ها واقعی و از API عمومی سایت‌ها: tgju، tsetmc و صرافی‌ها. فقط کتابخانه `requests` لازم است.
- دو ابزار آماده `get_json` و `save_report` را از فایل ابزارها کپی کن و اول نوت‌بوک اجرا کن.
- کنار هر عدد خروجی معلوم باشد ریال است یا تومان.

چه چیزی تحویل می‌دهی

۱ یک نوت‌بوک که از بالا تا پایین بدون خطا اجرا می‌شود.

۲ فایل گزارش `report.txt` که با `save_report` ساخته شده.

۳ یک بند کوتاه Markdown آخر نوت‌بوک: قانون سیگنالت چیست و چرا این عددها را انتخاب کردی.

نمره

گرفتن داده و تبدیل درست واحدها	۲۰
محاسبه‌ها با حلقه و دیکشنری	۳۰
قانون سیگنال و دلیلش	۲۰
گزارش خوانا	۱۵
کد تمیز: نام متغیر روشن، بدون تکرار بی‌دلیل	۱۵
بخش امتیازی	تا ۱۵

سه پروژه بعد از این قسمت؛ یکی را انتخاب کن

۱	**دیده‌بان نماد** امروز این چند سهم چه وضعی دارند؟ داده: tsetmc	پایه
۲	**گواهی سپرده و حباب طلا** طلا را کجا بخرم: گواهی، سکه یا آب‌شده؟ داده: tgju و صرافی‌ها	متوسط
۳	**کارنامه پرتفوی** سبد من کنار ۸ دارایی دیگر کجا ایستاده؟ داده: tsetmc و tgju	پیشرفته

سیگنال این پروژه‌ها **آموزشی** است، نه توصیه معامله. یک قانون ساده روی داده چندروزه فقط نشان می‌دهد چطور فکر را به کد تبدیل کنی.



سؤالِ این پروژه

پیش از شروع هر سه پروژه: داده را از کجا بیاورم و چه ابزاری لازم دارم؟

فهرست

شش فصل، دوازده مرحله؛ از «API یعنی چه؟» تا سلول تست

۰۱ — پیش از مرحله ۰
از داده تا سیگنال؛ نقشه هر سه پروژه — ۵
داستان، مسیر پنج‌مرحله‌ای، سه پروژه، چیزهایی که بلدی، قانون امتیاز

۰۲ — مرحله ۰ تا ۲
API، JSON و پنل Network — ۸
API یعنی چه، JSON یعنی چه، پنل Network، get_json، قیمت سکه، چاپ ساختار

۰۳ — مرحله ۳ تا ۵
تله‌های داده: متن، واحد، ترتیب و حرف عربی — ۱۲
متن به عدد، ریال و تومان، ترتیب روزها، «ی» و «ک» عربی، روزهای مشترک با set

۰۴ — مرحله ۶ تا ۸
چهار چیز تازه: try، مرتب کردن، جدول و فایل — ۱۵
try و except، لیست تاپل و sorted، f-string با عرض ثابت، save_report

۰۵ — مرحله ۹ و ۱۰
انحراف معیار و امتیاز z؛ بدون numpy — ۱۹
میانگین، انحراف معیار با حلقه، امتیاز z و معنی‌اش

۰۶ — مرحله ۱۱ و امتیازی
سلول تست، چک‌لیست و وقتی گیر می‌کنی — ۲۲
سلول تست، چک‌لیست آمادگی، مأموریت‌های امتیازی، جدول خطاهای رایج

فایل‌های همراه

`ابزارها.html`	دو ابزار آماده، چهار چیز تازه، حقه‌ها و جدول «اگر گیر کردی»
`راهنمای_API.html`	هر ۱۱ API با آدرس، نمونه واقعی جواب، کد و تله‌ها؛ و سلول تست

راهنمای خواندن این دفترچه

- **به زبان ساده** — همان مفهوم، بی‌اصطلاح
- **تله** — جایی که معمولاً خطا می‌کنی
- **سؤال فکری** — قبل از ادامه، کمی فکر کن
- **چیز تازه** — چیزی که در تمرین‌ها نبود
- **خروجی واقعی** — قالب کد

قسمت ۰ · پیش از شروع پروژه‌ها

داده
API عمومی tgju، tsetmc و چهار صرافی (نوبیتکس، والکس، بیت‌پین، تترلند)

خروجی
یک نوت‌بوک آماده با دو ابزار، که سلول تستش همه APIها را `OK` نشان می‌دهد

پیش‌نیاز
هفته ۲ (`set`)، هفته ۴ (JSON و پنل Network) و میان‌ترم

زمان پیشنهادی
یک جلسه، پیش از انتخاب پروژه

یک مسیر، دوازده مرحله، سه پروژه

به زبان ساده

قبل از این‌که پروژه‌ات را شروع کنی، یک بار جعبه‌ابزار را باز می‌کنیم: می‌فهمی سایت‌ها عددهایشان را از کجا می‌آورند، چطور همان عددها را با دو خط پایتون بگیری، کجاها معمولاً اشتباه می‌شود و چهار ابزار تازه‌ای که در تمرین‌ها نبود چطور کار می‌کنند. آخرش یک سلول تست اجرا می‌کنی که می‌گوید همه‌چیز آماده است.

API — گرفتن JSON
dict و list — مرتب کردن داده
محاسبه — بازده، حباب، ریسک
سیگنال — WAIT · WATCH · BUY
گزارش — report.txt

نقشه راه: مرحله ۰ تا ۱۱

۰ requests و get_json	۱ اولین API: سکه	۲ چاپ کن، حدس نزن	۳ متن به عدد، ریال به تومان	۴ «ی» و «ک» عربی
۵ روزهای مشترک با set	۶ except و try	۷ مرتب کردن نتیجه‌ها	۸ جدول و فایل گزارش	۹ انحراف معیار
۱۰ تمرین‌ها	۱۱ سلول تست			

از تمرین‌ها چه چیزی به کارت می‌آید

گرفتن JSON و رفتن داخل دیکشنری و لیست تودرتو	هفته ۴، هر دو بخش
پیدا کردن آدرس API در پنل Network	هفته ۴، بخش ۲
تبدیل متن به عدد با `replace` و `float` / `int`	هفته ۴، بخش ۱
اشتراک دو مجموعه با `set` و `&`	هفته ۲ و دفترهای میان‌ترم
f-string با دو رقم اعشار `:.2f`	میان‌ترم

چیزهای تازه

except / try
اگر یک سایت جواب نداد، برنامه نخوابد. «تازه ۱» فایل ابزارها.

مرتب کردن و گزارش
لیست تاپل با `sorted` ، f-string با عرض ثابت و نوشتن فایل؛ «تازه ۲ تا ۴».

آخر کار چه چیزی در دست داری

خروجی سلول تست · وقتی همه‌چیز آماده است

```
OK   tgju لحظه‌ای
OK   tgju تاریخچه
OK   tsetmc جست‌وجو
OK   tsetmc تاریخچه
OK   tsetmc حقیقی
OK   نوبیتکس
OK   والکس
OK   بیت‌پین
OK   تترلند
```



فصل اول · پیش از شروع

از داده تا سیگنال؛ نقشه هر سه پروژه

۰۱ فصل

می‌بینی سه پروژه چه مسیر مشترکی دارند، چه چیزهایی را از قبل بلدی و کدام پروژه برای تو مناسب‌تر است.

پیش از مرحله ۰ · ۴ دقیقه مطالعه

مسیر پروژه · API · سیگنال

سطح ۱ تمام شد. حالا وقت این است که همان لیست و دیکشنری و حلقه را روی داده واقعی بازار ایران امتحان کنی.

مأموریت این فصل

جعبه‌ابزار را آماده کن

قبل از این‌که سراغ سهم و طلا و سبد بروی، یک بار ابزارهایت را روی میز می‌چینی. آخر این دفترچه باید چهار کار از دستت بربیاید:

- آدرس داده هر سایتی را با پنل Network پیدا کنی
- جواب JSON را با `get_json` بگیری و تا عدد موردنظر پایین بروی
- از تله‌های داده (متن، واحد، ترتیب، حرف عربی) سالم رد شوی
- نتیجه‌ها را مرتب کنی و در یک فایل گزارش بنویسی

سطح ۱ دوره صفر تا کوآنت تمام شد. در CW و HW و میان‌ترم یاد گرفتی با `list` و `dict` و حلقه داده را مرتب کنی و رویش حساب کنی. حالا در پروژه پایانی همان کارها را روی **داده واقعی** انجام می‌دهی: عددهایی که همین امروز روی سایت بورس و tgju و صرافی‌ها هست. بدون numpy و بدون pandas؛ فقط با کدی که خودت می‌نویسی.

هر سه پروژه یک مسیر دارند

سؤال هر پروژه فرق دارد، ولی راهی که از داده خام به جواب می‌رسد همیشه همین پنج قدم است:

API — گرفتن JSON
dict و list — مرتب کردن داده
محاسبه — بازده، حباب، ریسک
سیگنال — WAIT · WATCH · BUY
گزارش — report.txt

قدم اول و آخر در هر سه پروژه تقریباً یکسان‌اند و همین دفترچه آن‌ها را یاد می‌دهد. قدم‌های وسط را هر پروژه به روش خودش می‌رود و دفترچه همان پروژه قدم‌به‌قدم توضیحشان می‌دهد.

سه پروژه، سه سطح

سه پروژه پایانی؛ هر کس یکی را انتخاب می‌کند

سطح	پروژه	سؤال اصلی	داده
۱ پایه	دیده‌بان نماد	امروز این چند سهم چه وضعی دارند و کدام ارزش زیر نظر گرفتن دارد؟	tsetmc
۲ متوسط	گواهی سپرده و حباب طلا	طلا را کجا بخرم: گواهی سپرده، سکه فیزیکی یا آب‌شده؟	tgju و صرافی‌ها
۳ پیشرفته	کارنامه پرتفوی	سبد من کنار صندوق طلا، دلار، سکه و بیت‌کوین کجا ایستاده؟	tsetmc و tgju

کدام را انتخاب کنم؟

اگر می‌خواهی مطمئن قدم برداری، پروژه ۱. اگر با سکه و طلا و دلار سروکار داری، پروژه ۲. اگر شارپ و ریسک و سبد برایت مهم است، پروژه ۳. پروژه ۳ یک **مسیر سبک** هم دارد: فقط رتبه‌بندی هشت دارایی، بدون سبد شخصی.

چه چیزهایی را از قبل بلدی

خبر خوب این است که تقریباً همه پروژه را قبلاً جایی نوشته‌ای. این جدول نشان می‌دهد هر تکه را کجا تمرین کردی:

آنچه در پروژه لازم است و جایی که تمرینش کردی

در پروژه لازم است	کجا تمرین کردی
گرفتن JSON با `get_json` و رفتن داخل دیکشنری تودرتو	هفته ۴، هر دو بخش
تبدیل متن به عدد با `replace`، ریال به تومان	هفته ۴ بخش ۱
ارزش ذاتی و حباب سکه	هفته ۴ سؤال ۱۷ تا ۱۹، دفتر ۳ میان‌ترم
ارزان‌ترین و گران‌ترین با الگوی `best = None`	هفته ۴ سؤال ۱۲، دفترهای میان‌ترم
بازده، سقف و کف، پول حقیقی، سیگنال	هفته ۴ بخش ۲، سؤال ۹ تا ۱۶
کندل، بدنه، گپ	دفتر ۷ میان‌ترم
افت از سقف و بازده سبد	دفترهای ۵ و ۶ میان‌ترم
اشتراک دو مجموعه با `set` و `&`	هفته ۲ و دفترهای میان‌ترم

فقط چهار چیز تازه است

`try` / `except`، مرتب کردن لیست نتیجه‌ها، جدول‌بندی گزارش با f-string و نوشتن فایل. هر چهار تا در فصل ۴ همین دفترچه آمده‌اند و هر کدام یک قالب آماده دارند.

قانون بازی: امتیاز و سطح

امتیاز مرحله‌های این دفترچه

امتیاز	سطح تو
۰ تا ۶۰	تازه‌وارد
۶۰ تا ۱۲۰	ابزارشناس
۱۲۰ تا ۱۶۰	آماده پروژه
بیش از ۱۶۰ (با امتیازی)	استادکار

هر مرحله کنار عنوانش امتیازی دارد. وقتی کد مرحله را اجرا کردی و خروجی‌ات شبیه نمونه همان مرحله شد، امتیازش مال توست. این امتیازها فقط نشان می‌دهند کجای راهی؛ جدول نمره اصلی پروژه در صفحه ۲ آمده.

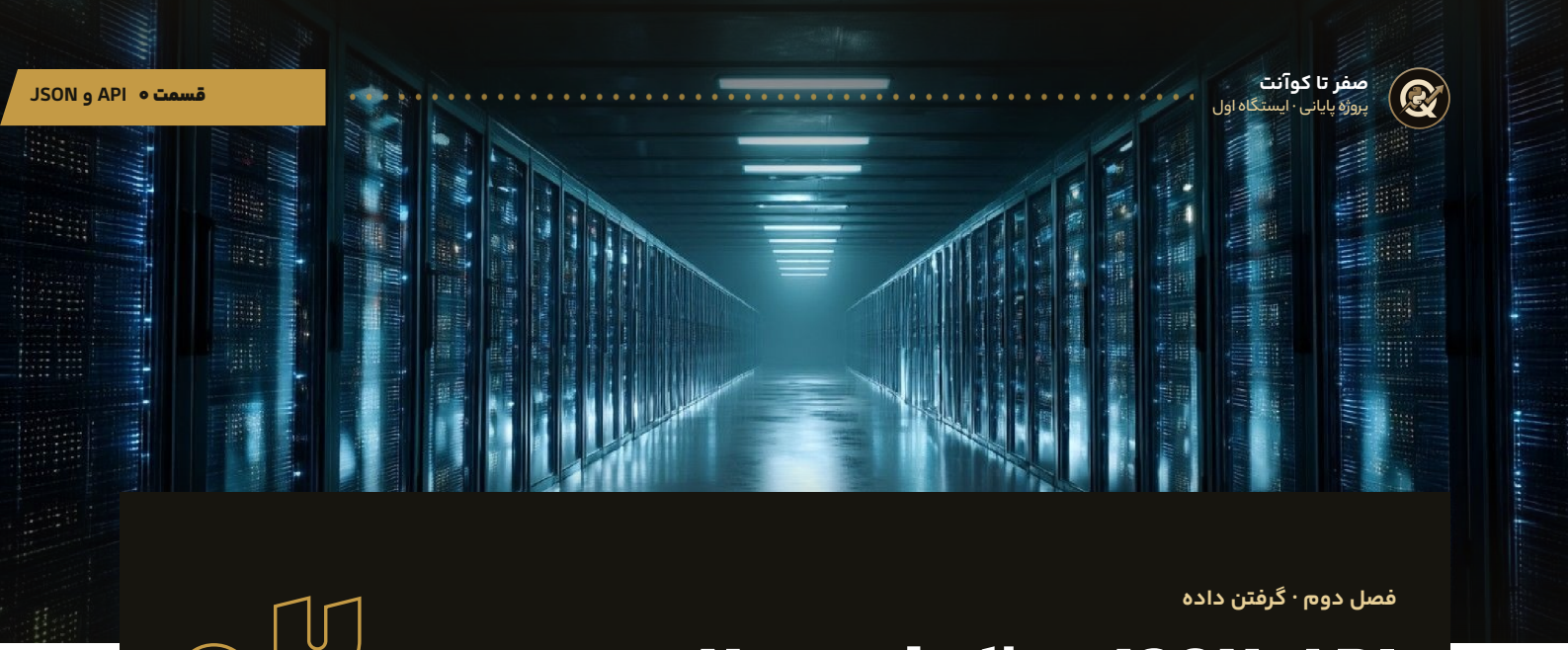
نشان گرفتی

نقشه‌خوان

مسیر را دیدی و می‌دانی کدام تکه‌ها را بلدی. حالا سراغ اولین ابزار برویم: API.

فصل بعد · API و JSON

API، JSON و پنل Network



فصل دوم · گرفتن داده

API، JSON و پنل Network

۰۲
فصل

می‌فهمی عددهای یک سایت از کجا می‌آیند، آدرسشان را پیدا می‌کنی و با get_json همان داده خام را در پایتون می‌گیری.

مرحله ۰ تا ۲ · ۴ دقیقه مطالعه

API | JSON | Network | requests

عددهایی که در صفحه سایت می‌بینی داخل خود صفحه نوشته نشده‌اند؛ صفحه آن‌ها را از یک آدرس دیگر می‌گیرد. آن آدرس را پیدا کن، کار تمام است.

مأموریت این فصل

اولین عدد واقعی

مأموریت این فصل ساده است: قیمت همین لحظه سکه بهار آزادی را، بدون این‌که سایت را باز کنی، با پایتون بگیری و چاپ کنی.

- آدرس داده tgju را پیدا کنی
- ابزار `get_json` را بسازی
- قدم‌به‌قدم تا کلید قیمت پایین بروی

API یعنی چه؟

وقتی صفحه اول tgju را باز می‌کنی، عددهای دلار و سکه و طلا داخل HTML صفحه نیستند. صفحه بعد از باز شدن، هر چند ثانیه از یک آدرس دیگر **داده خام** می‌گیرد و آن را مرتب نشان می‌دهد. آن آدرس را **API** می‌گویند.

مثل رستوران

در رستوران مستقیم به آشپزخانه نمی‌روی؛ به گارسون می‌گویی چه می‌خواهی و او غذا را می‌آورد. API همان گارسون است: یک آدرس را صدا می‌زنی و سایت، به‌جای یک صفحه خوشگل، داده خام به تو می‌دهد.

JSON یعنی دیکشنری و لیست

داده خام معمولاً **JSON** است، و JSON چیزی نیست جز همان دیکشنری و لیست تودرتو که در پایتون بلدی.

آکولاد `{}` یعنی دیکشنری، کروشه `[]` یعنی لیست. این تکه واقعی از جواب tgju است:

{} جواب tgju · کلید sekeb (سکه بهار آزادی)

```
{"current": {
   "sekeb": {"p": "2,595,100,000",
             "h": "2,595,100,000",
             "l": "2,594,700,000",
             "t": "۲۳ مهر"}
}}
```

یک دیکشنری که کلید `current` دارد؛ داخل `current` اسم هر دارایی؛ و داخل هر دارایی، کلید `p` که **قیمت فعلی** است. یعنی مسیر عدد ما این است: `["current"]["sekeb"]["p"]`.

پیدا کردن آدرس با پنل Network

همان کاری که در هفته ۴ برای tsetmc کردی:

1. صفحه را در Chrome باز کن، مثلاً `https://www.tgju.org/profile/sekeb/history`
2. `F12` بزن (مک: `Cmd + Option + I`) و تب **Network** را باز کن.
3. فیلتر **Fetch/XHR** را بزن تا فقط درخواست‌های داده بمانند.
4. صفحه را **رفرش** کن.
5. درخواستی را باز کن که اسمش شبیه داده است: `ajax.json`، `summary-table-data`، `ClosingPrice` و ...
6. تب **Preview** ساختار JSON را درختی نشان می‌دهد؛ تب **Headers** آدرس کامل (`Request URL`) را.
7. آدرس را در یک تب تازه باز کن. اگر JSON دیدی، همان را در پایتون صدا می‌زنی.

> **لازم نیست همه را خودت پیدا کنی**
> آدرس هر ۱۱ APIای که در سه پروژه لازم داری، با نمونه واقعی جواب و معنی همه کلیدها، در فایل `راهنمای_API.h` `tml` آمده. پنل Network را برای وقتی لازم داری که بخواهی داده‌ای بیرون از این فهرست بگیری.

مرحله ۰

۰ آماده‌سازی: requests و get_json

☆ ۱۰+ امتیاز

برای صدا زدن آدرس یک **کتابخانه** لازم داریم؛ یعنی جعبه‌ابزاری که دیگران نوشته‌اند و ما فقط `import` می‌کنیم. یک بار در ترمینال نصبش کن:

در پروژه ده‌ها بار آدرس صدا می‌زنی. به‌جای این‌که هر بار چند خط را تکرار کنی، یک بار تابع `get_json` را می‌سازی. این سلول را در **اولین سلول** نوت‌بوک بگذار و اجرا کن:

bash

```
pip install requests
```

python — **ابزار ۱: get_json · اولین سلول نوت‌بوک**

```
import requests

def get_json(url):
    response = requests.get(
        url,
        headers={"User-Agent": "Mozilla/5.0", "Accept": "application/json"},
        timeout=30,
    )
    response.raise_for_status()
    return response.json()
```

خط به خط

۱ کتابخانه `requests` را می‌آوریم.

۳ تابعی به اسم `get_json` می‌سازیم که یک ورودی می‌گیرد: آدرس (`url`).

۴–۸ آدرس را صدا می‌زنیم. `headers` به سایت می‌گوید «من مثل یک مرورگر معمولی‌ام و JSON می‌خواهم»؛ بعضی سایت‌ها بدون آن جواب نمی‌دهند. `timeout=30` یعنی بیشتر از ۳۰ ثانیه صبر نکن.

۹ اگر سایت خطا داد، همین‌جا خطا را نشان بده؛ نه این‌که بی‌صدا ادامه بدهیم.

۱۰ جواب را به دیکشنری و لیست پایتون تبدیل کن و برگردان.

اگر سلول را اجرا کنی و چیزی چاپ نشود، یعنی درست است: تابع فقط **ساخته** شده و هنوز کسی صدایش نزده.

مرحله ۱

۱ اولین API: قیمت سکه

☆ ۱۵+ امتیاز

حالا با یک خط آدرس tgju را صدا بزن و قیمت سکه بهار را چاپ کن:

python — **قالب مرحله ۱**

```
data = get_json("https://call1.tgju.org/ajax.json")
current = data["current"]
# To Do: قیمت سکه بهار (کلید "sekeb") را از current بردار و چاپ کن
```

نمونه خروجی (۲۳ مهر ۱۴۰۵)

```
2,595,100,000
```

⚠ این هنوز عدد نیست

چیزی که چاپ شد **متن** است، با ویرگول. با آن نمی‌شود حساب کرد. در فصل ۳ تبدیلش می‌کنیم.

مرحله ۲

۲ کلید را حدس نزن، چاپ کن

+۱۵ امتیاز

یک قانون طلایی: **هیچ‌وقت کلید را حدس نزن.** اول نوعش را ببین، بعد کلیدهایش را، بعد یک لایه پایین برو:

قدم‌به‌قدم تا عدد

```python
data = get_json("https://call1.tgju.org/ajax.json")
print(type(data))                    # dict یا list؟
print(list(data.keys()))             # کلیدهای سطح اول
print(type(data["current"]))         # یک لایه پایین‌تر
print(data["current"]["sekeb"])      # همه کلیدهای سکه بهار
```

نمونه خروجی

```
<class 'dict'>
['current', 'last', 'tolerance_low',
'tolerance_high']
<class 'dict'>
{'p': '2,595,100,000', 'h': '2,595,100,000', 'l':
'2,594,700,000', ...}
```

چه دیدیم

۲ جواب یک دیکشنری است، نه لیست.

۳ چهار کلید دارد؛ فقط `current` به کار ما می‌آید.

۵ داخل `sekeb` کلیدهای `p` (قیمت فعلی)، `h` و `l` (بیشترین و کمترین امروز)، `t` و `ts` (زمان آخرین قیمت) است.

سه قانون ادب با API

- بین درخواست‌ها، مخصوصاً در حلقه، کمی صبر کن: `imp ort time` و بعد `time.sleep(1)` .
- یک چیز را در حلقه ده بار نگیر؛ یک بار بگیر و در متغیر نگه دار.
- خطای `403` یا `429` یعنی زیاد درخواست زدی یا آدرس بسته است؛ چند دقیقه صبر کن.

نشان گرفتی

شکارچی API

اولین عدد واقعی را بدون باز کردن سایت گرفتی. حالا ببینیم این عدد کجاها می‌تواند فریبت بدهد.

فصل بعد · تله‌های داده

تله‌های داده: متن، واحد، ترتیب و حرف عربی



فصل سوم · داده تمیز

تله‌های داده: متن، واحد، ترتیب و حرف عربی

۰۳
فصل

عدد خام را به عدد درست تبدیل می‌کنی، ریال را از تومان جدا می‌کنی و چند سری قیمت را روی روزهای مشترک هم‌تراز می‌کنی.

مرحله ۳ تا ۵ · ۳ دقیقه مطالعه

set · ریال و تومان · replace

تقریباً همه یک بار در این تله‌ها می‌افتند. فرقش این است که تو از قبل می‌دانی کجا هستند.

مأموریت این فصل

بازرس داده

داده خام همیشه آن‌طور که فکر می‌کنی نیست. در این فصل یاد می‌گیری قبل از هر حسابی، داده را بازرسی کنی:

- متن با ویرگول را به `int` یا `float` تبدیل کنی
- بدانی هر عدد ریال است یا تومان، و هر لیست از کدام سر شروع می‌شود
- اسم‌ها را درست مقایسه کنی و روزهای مشترک چند سری را پیدا کنی

مرحله ۳

۳ متن به عدد، ریال به تومان

☆ ۱۵+ امتیاز

تله اول: قیمت‌ها متن‌اند. tgju قیمت را مثل `"2,595,100,000"` می‌دهد: یک رشته با ویرگول. اول ویرگول را با `replace` حذف کن، بعد `int` یا `float`. انس طلا اعشار دارد، پس `float`:

متن به عدد python

```python
text = current["sekeb"]["p"]                    # "2,595,100,000"
price_rial = int(text.replace(",", ""))         # ریال، عدد صحیح
price_toman = price_rial // 10                  # تومان
ounce = float(current["ons"]["p"].replace(",", ""))    # دلار، اعشاری
print(price_rial, price_toman, ounce)
```

نمونه خروجی

```
2595100000 259510000 4171.37
```

تله دوم: ریال و تومان. tgju و tsetmc و نوبیتکس لحظه‌ای ریال می‌دهند؛ والکس و بیت‌پین و تترلند تومان. اگر عددت ده برابر غلط است، تقریباً همیشه همین است.

تله سوم: ترتیب. tgju و tsetmc روزها را از **جدید به قدیم** می‌دهند. یعنی `[0]` تازه‌ترین روز است، نه قدیمی‌ترین.

جمع‌بندی واحدها و ترتیب‌ها (از راهنمای API)

API	واحد قیمت	نوع قیمت	ترتیب لیست
tgju لحظه‌ای	ریال (انس: دلار)	متن با ویرگول	–
tgju تاریخچه	ریال (انس: دلار)	متن با ویرگول	جدید ← قدیم
tsetmc	ریال	عدد	جدید ← قدیم
نوبیتکس لحظه‌ای	ریال	متن	–
نوبیتکس تاریخچه	تومان	عدد	قدیم ← جدید
والکس و بیت‌پین	تومان	متن	–
تترلند	تومان	عدد	–

⚠ tsetmc اعشار می‌گذارد

tsetmc قیمت را `4540.0` می‌دهد، نه `4540` . قبل از چاپ `int()` بزن تا `.0` نیاید.

مرحله ۴

۴ «ی» و «ک» عربی

☆ ۱۰+ امتیاز

یک تله بامزه در بورس: سایت tsetmc «ی» و «ک» را **عربی** می‌نویسد (`ي` و `ك`). در صفحه هیچ فرقی نمی‌بینی، ولی برای پایتون این‌ها دو حرف کاملاً متفاوت‌اند. یعنی «فملی» که تو تایپ می‌کنی با «فملی» که سایت برمی‌گرداند مساوی نیست:

مقایسه دو «فملی» python

```python
print("فملي" == "فملی")          # اولی «ی» عربی دارد
name = "فملي ".replace("ي", "ی").replace("ك", "ک").strip()
print(name == "فملی")
```

خروجی

```
False
True
```

راه‌حل دو `replace` است، یکی برای «ی» و یکی برای «ک»، و یک `strip` برای فاصله‌های اضافه. در پروژه ۱ و ۳ این خط را روی هر اسمی که از tsetmc می‌آید بزن:

حقه ۳ فایل ابزارها

```
name = item["lVal18AFC"].replace("ي",
"ی").replace("ك", "ک").strip()
```

مرحله ۵

۵ روزهای مشترک با set

+۱۵ امتیاز

وقتی چند سری قیمت داری، روزهایشان یکی نیست. بورس پنج‌شنبه و جمعه تعطیل است، گواهی سپرده فقط روزهای کاری بورس کالا معامله می‌شود، ولی انس و بیت‌کوین هر روز قیمت دارند. برای حساب درست، همه قیمت‌ها باید مال **همان روز** باشند.

راه‌حلش را از هفته ۲ بلدی: کلیدهای تاریخ هر سری را `set` کن و با `&` اشتراکشان را بگیر. آخر هم `sorted` تا مرتب شوند:

قالب مرحله ۵ python

```
# عددهای نمونه؛ در پروژه از API تاریخچه می‌آیند
coin = {"1405/07/19": 2580, "1405/07/20": 2585, "1405/07/21": 2590, "1405/07/23": 2595}
ounce = {"1405/07/20": 4150, "1405/07/21": 4160, "1405/07/22": 4165, "1405/07/23": 4171,
"1405/07/24": 4180}

common = set(coin) & set(ounce)
days = sorted(common)
print(days)
# To Do: برای هر روز در days، قیمت هر دو سری را کنار هم چاپ کن
```

خروجی

```
['1405/07/20', '1405/07/21', '1405/07/23']
```

چرا sorted روی تاریخ شمسی درست کار می‌کند؟

چون ماه و روز همیشه دو رقمی‌اند (`07` ، نه `7`). پس مرتب کردن متنی همان مرتب کردن زمانی است.

سؤال فکری

اگر قیمت گواهی سپرده مال چهارشنبه باشد و انس مال جمعه، حبابی که حساب می‌کنی درست است؟ کلید `ts` هر دارایی در tgju به تو می‌گوید هر عدد مال کی است.

نشان گرفتی

بازرس داده

متن را به عدد تبدیل کردی، واحدها را جدا کردی و سری‌ها را هم‌تراز کردی. حالا داده‌ات آماده حساب است.



فصل چهارم · چیزهایی که در تمرین‌ها نبود

چهار چیز تازه: try، مرتب کردن، جدول و فایل

۰۴
فصل

برنامه‌ات با قطع شدن یک سایت نمی‌خوابد، نتیجه‌ها را از زیاد به کم مرتب می‌کنی و گزارش را تمیز در یک فایل می‌نویسی.

مرحله ۶ تا ۸ · ۳ دقیقه مطالعه

try/except · sorted · f-string · open

بقیه پروژه همان چیزهایی است که نوشته‌ای. فقط این چهار تا تازه‌اند، و هر کدام یک قالب آماده دارد: کپی کن و فقط اسم‌ها را عوض کن.

مأموریت این فصل

سه ابزار آخر جعبه

تا این‌جا داده را گرفتی و تمیز کردی. سه کار مانده که در هر سه پروژه لازم است:

- اگر یک سایت جواب نداد، برنامه ادامه بدهد
- نتیجه‌ها از بهترین به بدترین مرتب شوند
- گزارش به‌صورت یک جدول خوانا در `report.txt` ذخیره شود

مرحله ۶

۱: تازه try و except

۱۵+ امتیاز

وقتی یک سایت جواب نمی‌دهد، `get_json` خطا می‌دهد و **کل برنامه می‌ایستد**. `try` یعنی «امتحان کن». `except` یعنی «اگر خطا شد، به‌جایش این کار را بکن»:

قالب except / try

```python
prices = {}

try:
    q = get_json("https://service.tetherland.com/api/v4/currencies")
    prices["Tetherland"] = q["price"]
except Exception:
    print("تترلند جواب نداد، رد شدم")

# برنامه این‌جا ادامه می‌دهد، چه تترلند جواب داده باشد چه نه
print(prices)
```

خط به خط

- **۳** از این‌جا به بعد را «امتحان» کن.
- **۴–۵** آدرس را صدا بزن و قیمت را در دیکشنری بگذار. اگر هر کدام از این دو خط خطا بدهد، پایتون مستقیم به `except` می‌پرد.
- **۶–۷** اگر خطا شد، فقط یک پیام چاپ کن و برو جلو.
- **۱۰** این خط همیشه اجرا می‌شود.

هر سایت، یک try جدا

اگر همه صرافی‌ها را در یک `try` بگذاری، خطای اولی بقیه را هم می‌پراند. هر کدام را در `try` خودش بگذار، و آخر کار میانگین را فقط از آن‌هایی بگیر که جواب داده‌اند.

مرحله ۷

۷ تازه ۲: مرتب کردن نتیجه‌ها

+۱۵ امتیاز

`sorted` را روی `set` و رشته زدی، ولی روی **لیست دیکشنری** کار نمی‌کند. حقه‌اش این است: یک **لیست تاپل** بساز. `sorted` تاپل‌ها را اول بر اساس **عضو اولشان** مرتب می‌کند:

sorted روی لیست تاپل

```python
pairs = [(2, "فولاد"), (3, "فملی"), (1, "وبملت")]
print(sorted(pairs))
```

خروجی

```
[(1, 'وبملت'), (2, 'فولاد'), (3, 'فملی')]
```

ولی ما معمولاً **از زیاد به کم** می‌خواهیم: بهترین امتیاز اول. حقه دوم: عدد را منفی کن، و موقع چاپ دوباره منفی‌اش کن تا مثبت شود:

قالب مرحله ۷

```python
results = [{"symbol": "فولاد", "score": 2},
           {"symbol": "فملی", "score": 3},
           {"symbol": "وبملت", "score": 1}]

pairs = []
for r in results:
    pairs.append((-r["score"], r["symbol"]))
for neg_score, symbol in sorted(pairs):
    print(symbol, -neg_score)
```

خروجی

```
فملی 3
فولاد 2
وبملت 1
```

راه دوم، که بلدی

با الگوی `best = None` بزرگ‌ترین را پیدا کن، چاپش کن، از لیست حذفش کن و تکرار کن. کندتر است ولی کاملاً قبول است.

مرحله ۸

۸ تازه ۳ و ۴: جدول گزارش و فایل

☆ ۱۵+ امتیاز

`f"{x:.2f}"` را در میان‌ترم دیدی. چند گزینه دیگر از همان خانواده، برای این‌که ستون‌های گزارش صاف زیر هم بیایند:

گزینه‌های f-string

نوشتن	یعنی	مثال خروجی
`f"{x:,}"`	ویرگول هزارگان	`2,595,100,000`
`f"{x:,.2f}"`	ویرگول و دو رقم اعشار	`4,171.37`
`f"{name:<10}"`	متن ۱۰ خانه، چپ‌چین	`فولاد`
`f"{x:>12,}"`	عدد ۱۲ خانه، راست‌چین با ویرگول	`2,595,100`

اگر جدول‌بندی اذیتت کرد

ساده بنویس و ستون‌ها را با `|` جدا کن؛ نمره کم نمی‌شود: `f"{symbol} | {close:,} | {change:.2f}% | {signal}"`

گزارش آخر کار باید در یک **فایل** ذخیره شود. این هم یک تابع کوچک است که یک بار می‌سازی و کنار `get_json` در اولین سلول می‌گذاری:

ابزار ۲: save_report · اولین سلول نوت‌بوک

```python
def save_report(lines, filename):
    with open(filename, "w", encoding="utf-8") as f:
        for line in lines:
            f.write(line + "\n")
    print("گزارش ذخیره شد:", filename)
```

خط به خط

۱ دو ورودی: لیست خط‌ها و اسم فایل.

۲ فایل را باز می‌کند. `"w"` یعنی از نو بنویس (`"a"` یعنی به ته فایل اضافه کن). `utf-8` لازم است تا فارسی خراب نشود.

۳–۴ با یک حلقه هر خط را می‌نویسد؛ `"\n"` یعنی برو سر خط بعد.

۵ خبر می‌دهد که فایل ساخته شد.

روش استفاده: هر خطی که می‌خواهی در گزارش باشد را به یک لیست `append` کن و آخر کار لیست را به `save_report` بده:

قالب مرحله ۸

```python
report = []
report.append("نتیجه امروز")
for neg_score, symbol in sorted(pairs):
    # To Do: یک خط با f-string بساز: نماد ۸ خانه چپ‌چین، امتیاز ۴ خانه راست‌چین
    pass
save_report(report, "report.txt")
```

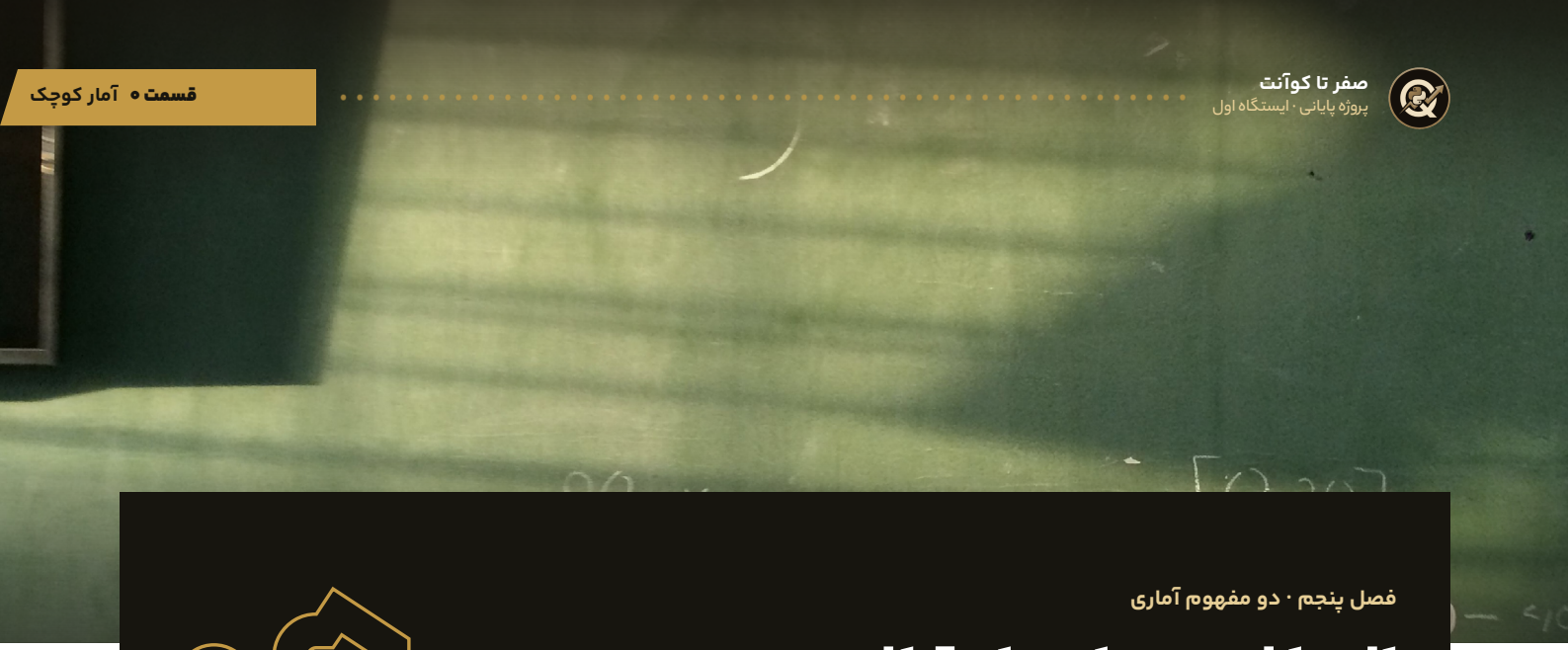
نمونه report.txt

```
نتیجه امروز
فملی          3
فولاد         2
وبملت         1
```

نشان گرفتی

گزارش‌نویس

برنامه‌ات حالا با قطع شدن یک سایت نمی‌خوابد، نتیجه‌ها را مرتب می‌کند و گزارش را در فایل می‌نویسد.



فصل پنجم · دو مفهوم آماری

انحراف معیار و امتیاز z؛ بدون numpy

فصل ۵

با یک حلقه حساب می‌کنی عددها معمولاً چقدر از میانگین دورند، و می‌فهمی امروز چقدر از حالت عادی فاصله دارد.

مرحله ۹ و ۱۰ · ۳ دقیقه مطالعه

میانگین · انحراف معیار · امتیاز z

پروژه ۲ با امتیاز z حباب طلا را می‌سنجد و پروژه ۳ با انحراف معیار ریسک سبد را. هر دو را با همان حلقه‌ای که بلدی حساب می‌کنی.

مأموریت این فصل

یک خط‌کش برای «عادی»

«امروز گران است» یعنی چه؟ گران‌تر از چه؟ در این فصل دو عدد می‌سازی که به این سؤال جواب می‌دهند:

- **انحراف معیار:** عددها معمولاً چقدر بالا و پایین می‌روند
- **امتیاز z:** امروز چند برابر آن نوسان معمول از میانگین دور است

مرحله ۹

۹ انحراف معیار با حلقه

☆ ۱۵+ امتیاز

انحراف معیار یعنی «عددها به‌طور معمول چقدر از میانگین دورند». یک مثال دستی: بازده‌های روزانه `[1, -1, 3, 1]` درصد.

انحراف معیار قدم‌به‌قدم

قدم	حساب	نتیجه
میانگین	(۱ + −۱ + ۳ + ۱) ÷ ۴	۱
فاصله از میانگین	۰، −۲، ۲، ۰	
مجذور فاصله‌ها	۰، ۴، ۴، ۰	جمع: ۸
واریانس	۸ ÷ (۴ − ۱)	۲٫۶۷
انحراف معیار	ریشه ۲٫۶۷	**۱٫۶۳**

یعنی بازده این سهم معمولاً حدود ۱٫۶ درصد بالا و پایین میانگین است.

همین را با حلقه بنویس. ریشه در پایتون یعنی **به توان ۰٫۵**:

حقه ۱ فایل ابزارها: انحراف معیار python

```python
returns = [1, -1, 3, 1]
mean = sum(returns) / len(returns)

variance = 0
for r in returns:
    variance += (r - mean) ** 2
variance = variance / (len(returns) - 1)
std = variance ** 0.5

print(round(mean, 2), round(variance, 2), round(std, 2))
```

خروجی

```
1.0 2.67 1.63
```

خط به خط

- **۲** میانگین: جمع تقسیم بر تعداد.
- **۴–۶** برای هر عدد، فاصله‌اش از میانگین را به توان ۲ می‌رسانیم و جمع می‌زنیم.
- **۷** تقسیم بر «تعداد منهای یک»؛ این همان نسخه‌ای است که برای نمونه داده استفاده می‌شود.
- **۸** ریشه دوم با `** 0.5` .

چرا تقسیم بر n − ۱؟

وقتی از روی چند روز داده نوسان کل بازار را تخمین می‌زنی، تقسیم بر n نوسان را کمی کمتر از واقعیت نشان می‌دهد. تقسیم بر n − ۱ این کم‌برآوردی را جبران می‌کند. برای پروژه کافی است همین فرمول را به کار ببری.

مرحله ۱۰

۱۰ امتیاز z

☆ ۱۰+ امتیاز

امتیاز z می‌گوید امروز **چند برابر نوسان معمول** از حالت عادی دور است:

فرمول

```
z = (today - mean) / std
```

مثال: میانگین حباب سکه در صد روز گذشته ۵ درصد بوده و انحراف معیارش ۲ درصد. امروز حباب ۹ درصد است:

قالب مرحله ۱۰ python

```python
mean_bubble = 5      # درصد
std_bubble = 2       # درصد
today = 9            # درصد
# To Do: z را حساب و چاپ کن، و با if بگو امروز «غیرعادی گران» است یا نه
```

خروجی

```
z = 2.0
```

یعنی امروز حباب دو برابر نوسان معمول از میانگینش بالاتر است: **غیرعادی گران**.

یک خط‌کش ساده برای z

z	معنی
بین ۱− و ۱	عادی
بیشتر از ۲	غیرعادی بالا (گران)
کمتر از ۲−	غیرعادی پایین (ارزان)

سؤال فکری

اگر انحراف معیار صفر باشد (همه روزها یک عدد) چه می‌شود؟ قبل از تقسیم، با `if` چه چیزی را چک می‌کنی؟

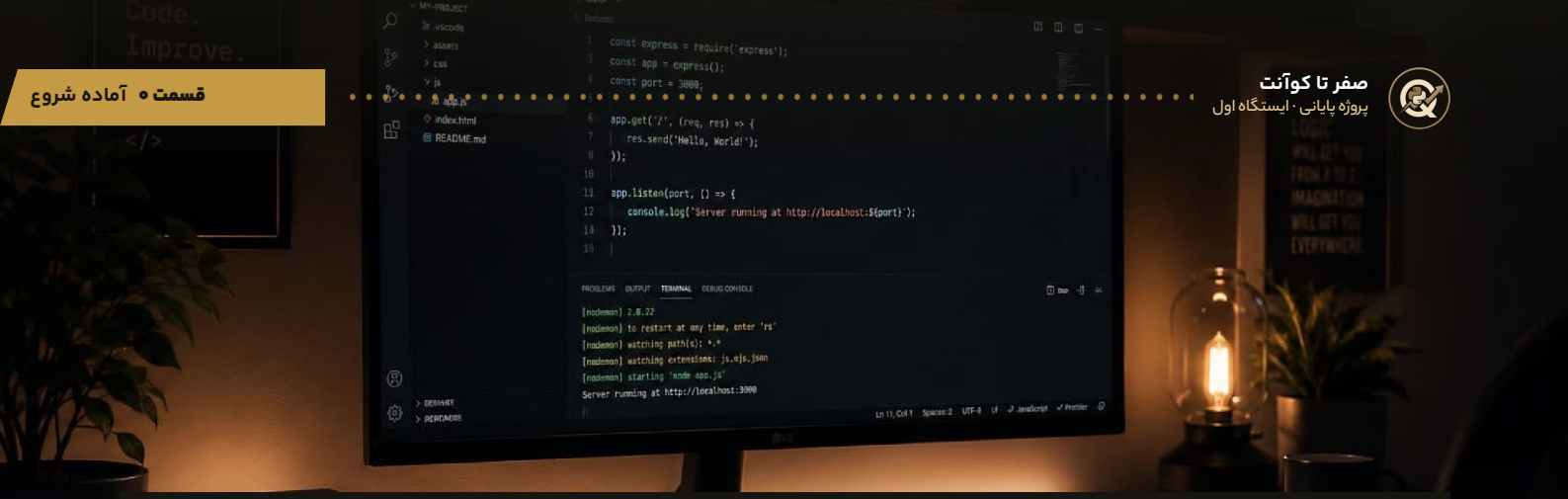
نشان گرفتی

خط‌کش‌ساز

حالا می‌توانی بگویی یک عدد «عادی» است یا نه. پروژه ۲ و ۳ روی همین دو عدد ساخته می‌شوند.

فصل بعد · آماده شروع

سلول تست، چک‌لیست و وقتی گیر می‌کنی



فصل ششم · آخرین چک

۰۶ سلول تست، چک‌لیست و وقتی گیر می‌کنی

یک بار همه APIها را صدا می‌زنی، جعبه‌ابزار را چک می‌کنی و جدول خطاهای رایج را کنار دستت می‌گذاری.

فصل

مرحله ۱۱ و امتیازی ۴ دقیقه مطالعه

سلول تست | دیباگ | تحویل

اگر سلول تست همه را OK نشان داد، مشکل هرچه باشد از کد توست، نه از سایت. این خودش نصف دیباگ است.

مأموریت این فصل

آخرین چک پیش از پروژه

سه کار مانده تا جعبه‌ابزارت کامل شود:

- سلول تست را اجرا کنی و ببینی همه APIها جواب می‌دهند
- خانه‌های چک‌لیست آمادگی را تیک بزنی
- یکی از سه پروژه را انتخاب کنی و دفترچه‌اش را باز کنی

مرحله ۱۱

۱۱ سلول تست

+۱۰ امتیاز

این سلول آخر فایل `راهنمای_API.html` است. همه APIهای سه پروژه را یک بار صدا می‌زند و می‌گوید کدام جواب داد. بعد از سلول ابزارها اجرایش کن:

سلول تست python

```python
import time

tests = [
    ("tgju لحظه‌ای", "https://call1.tgju.org/ajax.json"),
    ("tgju تاریخچه", "https://api.tgju.org/v1/market/indicator/summary-table-data/sekeb?length=5"),
    ("tsetmc جست‌وجو", "https://cdn.tsetmc.com/api/Instrument/GetInstrumentSearch/فولاد"),
    ("tsetmc تاریخچه", "https://cdn.tsetmc.com/api/ClosingPrice/GetClosingPriceDailyList/46348559193224090/5"),
    ("tsetmc حقیقی", "https://cdn.tsetmc.com/api/ClientType/GetClientTypeHistory/46348559193224090"),
    ("نوبیتکس", "https://apiv2.nobitex.ir/market/stats?srcCurrency=usdt&dstCurrency=rls"),
    ("والکس", "https://api.wallex.ir/hector/web/v1/markets?version=2"),
    ("بیت‌پین", "https://api.bitpin.ir/v4/mkt/prices/"),
    ("تترلند", "https://service.tetherland.com/api/v4/currencies"),
]

for name, url in tests:
    try:
        get_json(url)
        print("OK   ", name)
    except Exception as error:
        print("FAIL ", name, "→", error)
    time.sleep(1)
```

چه چیزهایی را می‌شناسی

۳–۱۳ یک **لیست تاپل**: هر تاپل اسم API و آدرسش.

۱۵ حلقه روی تاپل‌ها؛ هر تاپل را مستقیم در دو متغیر باز می‌کنیم.

۱۶–۲۰ همان `try` / `except` فصل ۴: اگر یک سایت قطع بود، بقیه تست می‌شوند.

۲۱ یک ثانیه صبر بین درخواست‌ها؛ قانون ادب.

اگر یکی FAIL شد

اول آدرس را در مرورگر باز کن. اگر در مرورگر هم باز نشد، مشکل از سایت یا اینترنت است، نه کد تو. اگر فیلترشکن روشن است خاموشش کن؛ بعضی سایت‌های ایرانی IP خارجی را می‌بندند.

چک‌لیست آمادگی

- [ ] `requests` نصب است و `get_json` و `save_report` در اولین سلول نوت‌بوک اجرا شده‌اند
- [ ] قیمت سکه بهار را از tgju گرفته‌ام و به عدد و تومان تبدیل کرده‌ام
- [ ] قبل از نوشتن هر کلید، `type` و `keys` را چاپ می‌کنم
- [ ] می‌دانم کدام API ریال می‌دهد و کدام تومان، و `[0]` کدام روز است
- [ ] «ی» و «ک» عربی را با `replace` درست می‌کنم
- [ ] روزهای مشترک دو سری را با `set` و `&` پیدا کرده‌ام
- [ ] `except` / `try` ، مرتب کردن با لیست تاپل و f-string با عرض ثابت را امتحان کرده‌ام
- [ ] سلول تست همه را `OK` نشان می‌دهد و پروژه‌ام را انتخاب کرده‌ام

بخش امتیازی

۱ **مأموریت ۱ · کاوشگر کلید (+۱۰ XP):** با یک حلقه روی `current` در جواب tgju، همه کلیدهایی را که `"ime_certificate"` در اسمشان هست با قیمتشان چاپ کن.

۲ **مأموریت ۲ · میانگین صرافی‌ها (+۱۵ XP):** قیمت تتر را از چهار صرافی، هر کدام در `try` جدا، بگیر، همه را تومان کن و میانگین آن‌هایی را که جواب دادند چاپ کن.

۳ **مأموریت ۳ · نوسان سکه (+۱۵ XP):** تاریخچه ۳۰ روز سکه بهار را از tgju بگیر، بازده روزانه بساز و انحراف معیارش را با حلقه حساب کن.

یک عادت خوب

هر جا گیر کردی، قبل از هر کاری **چاپ کن**: نوع داده (`type`)، طول (`len`) و کلیدها (`keys`). نصف خطاهای پروژه با یک `print` درست سر جایش حل می‌شود.

حالا کدام دفترچه؟

پروژه‌ات را انتخاب کن و دفترچه‌اش را باز کن: دیده‌بان نماد (پایه)، گواهی سپرده و حباب طلا (متوسط) یا کارنامه پرتفوی (پیشرفته). فایل‌های `ابزارها.html` و `راهنمای_API.html` در هر سه پروژه کنار دستت هستند.

نشان گرفتی

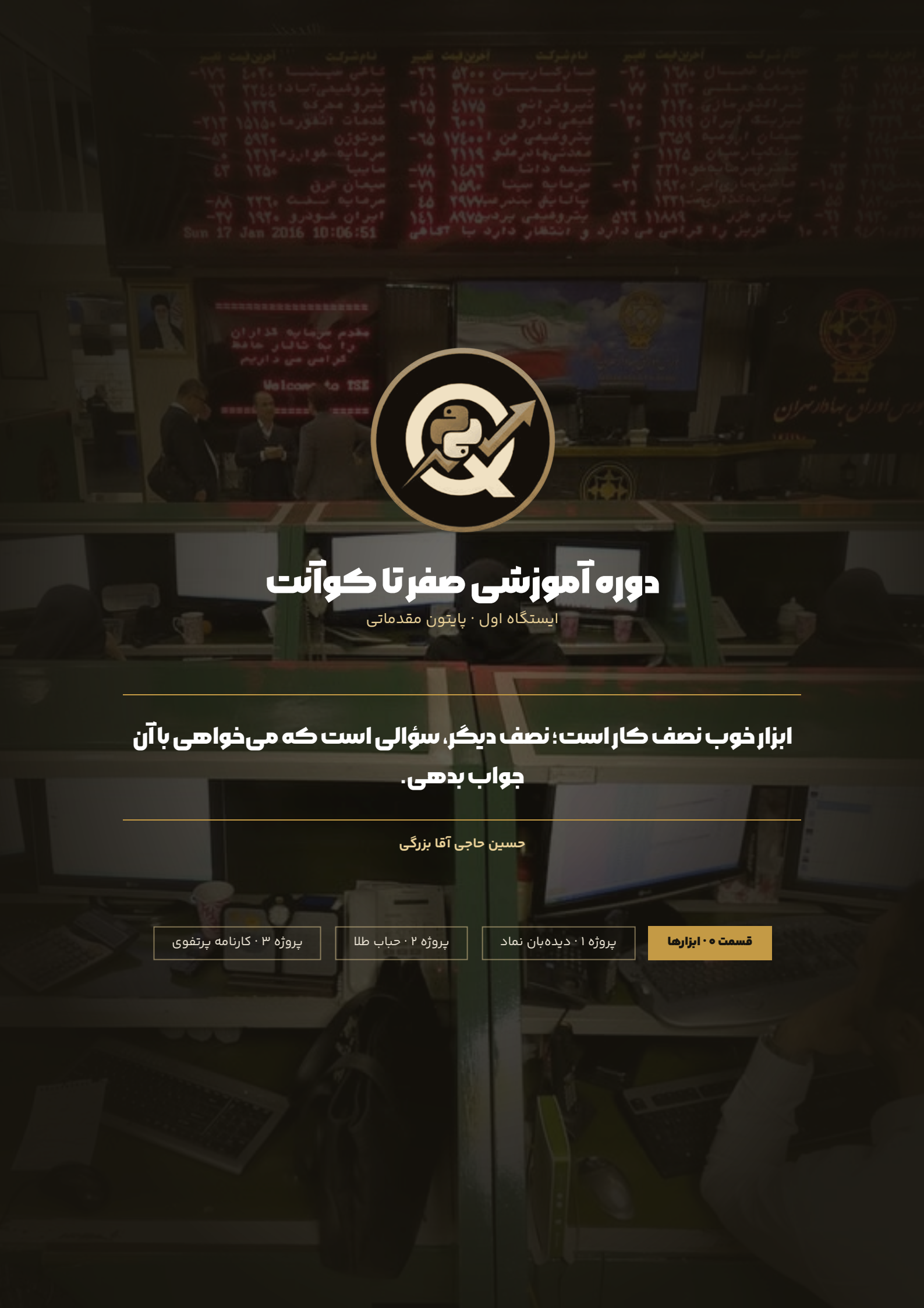
آماده پروژه

جعبه‌ابزارت کامل است. با ۱۶۰ امتیاز مرحله‌ها «آماده پروژه» هستی؛ هر مأموریت امتیازی تو را «استادکار» می‌کند. قسمت بعد: پروژه ۱، دیده‌بان نماد.

اگر گیر کردی

خطا	معمولاً یعنی	چه کنم
`KeyError: 'xxx'`	کلیدی که نوشتی در دیکشنری نیست	`print(list(data.keys()))` ؛ حرف بزرگ و کوچک، «ی» عربی و فارسی
`TypeError: 'str' and 'int'`	با متن حساب کردی	`replace(",", "")` و بعد `int()` یا `float()`
`ValueError: invalid literal for int()`	متن اعشار دارد، مثل `"4,171.37"`	`float` بزن، نه `int`
`IndexError`	لیست کوتاه‌تر از چیزی است که فکر می‌کردی	`print(len(...))`
`ZeroDivisionError`	تقسیم بر صفر	قبلش با `if` چک کن
`HTTPError: 403 / 429`	سایت تو را بست یا زیاد درخواست زدی	فیلترشکن را خاموش کن، صبر کن، `time.sleep(1)`
`ConnectionError / Timeout`	اینترنت یا سایت قطع است	آدرس را در مرورگر باز کن
عدد ۱۰ برابر غلط	ریال و تومان قاطی شده	جدول واحدها در فصل ۳
ترتیب روزها برعکس	tgju و tsetmc جدید به قدیم می‌دهند	`[0]` تازه‌ترین روز است

موفق باشی!



دوره آموزشی صفر تا کوآنت

ایستگاه اول · پایتون مقدماتی

ابزار خوب نصف کار است؛ نصف دیگر، سؤالی است که می‌خواهی با آن جواب بدهی.

حسین حاجی آقا بزرگی

قسمت ۰۰ ابزارها | پروژه ۰۱ · دیده‌بان نماد | پروژه ۰۲ · حباب طلا | پروژه ۰۳ · کارنامه پرتفوی